

aspgen Generation Decision Guide

Flags, generated trees, dependency direction, code shapes, and selection rules for Architecture Theta.

Choose the profile first. Generate only the architecture your product needs.

Version 1.0 | Architecture Theta | 03 August 2026

1. Fast decision map

If you need...	Choose	Start with
A classic CRUD API and desktop client quickly	Simple fullstack	--app fullstack --simple
Business rules, CQRS, repositories, and clean boundaries	DDD fullstack	--app fullstack --backend ddd
A server only with Clean Architecture	Clean Web API	--app webapi
A Prism desktop application first	WPF	--app wpf
A local DDD desktop application without HTTP	Local DDD WPF	--app wpf --backend ddd
A Renoir-style DDD/Blazor application	Blazor	--app blazor
Development records at startup	Dummy seed	--seed:dummy with --simple or --backend ddd
Fluent WPF controls and theme resources	WPF-UI	--theme:wpfui
Add desktop later without regenerating the server	Incremental UI	add ui --project PATH

Rule: --simple and --backend ddd are mutually exclusive. --seed dummy requires one of those backend profiles.

2. Flag and command reference

Required generation order

Run new before any add command. The new command creates the project directory, solution, source projects, and .aspgen/manifest.json; incremental commands use that manifest to determine which architecture and files already exist.

```
aspgen new Library --app fullstack --simple --output ./Library
aspgen add entity Book title:string author:string pages:int published:date available:bool
--project ./Library
```

If ./Library does not exist, add entity fails by design. When --project is omitted, aspgen searches the current directory and its parents for the manifest.

new NAME

Creates the initial project, manifest, solution, projects, references, and host files.

```
aspgen new NAME --app webapi|wpf|blazor|fullstack [options]
```

Option	Meaning	Changes the generated graph	
--app webapi	Server only	Domain, Application, Infrastructure, WebApi	
--app wpf	Desktop only	Prism/Dryloc Desktop project	
--app blazor	Renoir-style web UI	DomainModel, Application, Infrastructure, Persistence, Resources, AppBlazor	
--app fullstack	Server + desktop	Backend graph plus Desktop project	
--output PATH	Destination folder	Writes the whole solution below PATH	
--simple	Active Record-style CRUD	One WebApi project; no DDD/CQRS layers	
--backend ddd	DDD/CQRS backend	Clean layers, repositories, handlers, validators	
--app wpf --backend ddd	Local DDD desktop	Domain, Application, Infrastructure/SQLite, Desktop; no WebApi	
--database sqlite	postgres`	Persistence provider	SQLite is the default; PostgreSQL is opt-in
--seed dummy	Development data	Startup seeder plus entity sample records	
--theme wpfui	WPF-UI theme	WPF-UI package, resources, FluentWindow shell	
--templates PATH	Template override root	Uses editable templates when a matching group exists	
--dry-run	Preview	Prints intended writes without creating files	
--force	Allow overwrite	Replaces existing generated files where supported	

add entity NAME

Adds a typed entity or vertical CRUD slice. Property syntax follows the Rails-style convention:

```
aspgen add entity Person name:string age:int born:date active:bool --project ./MyApp
```

Supported types include string, int, long, decimal, float, bool, date, datetime, guid, and nullable forms such as string? or int?.

Existing profile	Entity output
Clean Web API	Domain entity only; use --backend ddd for CRUD infrastructure
Simple Web API	EF model, DbSet, direct Minimal API CRUD
DDD Web API	Domain entity, repository, CQRS CRUD, validation, endpoint
WPF/fullstack	Adds the matching Prism module, forms, ViewModel, and store
Seed enabled	Adds three deterministic typed records to DatabaseSeeder.cs

Other incremental commands

Command	Intended profile	Creates or updates	
add ui --framework wpf	Web API/fullstack	Desktop project and solution membership	
add module NAME	WPF/fullstack	Prism module, views, ViewModel, module catalog	
add feature NAME fields...	Non-simple Web API	Request/response records, handler, validator, endpoint	
add context NAME	Blazor/Renoir	Bounded context manifest entry	
add aggregate NAME fields...	Blazor/Renoir	Aggregate root, validation, CRUD service/page	
add value-object NAME fields...	Blazor/Renoir	Immutable value object	
add domain-service NAME	Blazor/Renoir	Stateless domain policy	
add repository NAME --aggregate NAME	Blazor/Renoir	Aggregate repository contract	
add event NAME fields...	Blazor/Renoir	Immutable domain event	
`add database sqlite	postgres`	Web API/fullstack	Database component marker; existing contexts are preserved
add service NAME	Non-simple Web API	Application service contract	

3. Dependency trees

Clean Web API

```
WebApi
-> Infrastructure
-> Application
-> Domain
```

The dependency arrow points toward policy. Domain does not depend on any other generated project.

Simple fullstack

```
Desktop (Prism / MVVM)
-> HTTP API
-> WebApi
-> EF Core / SQLite (default) or PostgreSQL
```

The simple profile intentionally collapses layers to reduce ceremony. Its safety comes from a small surface, typed models, and direct CRUD endpoints.

DDD fullstack

```
Desktop module / HTTP endpoint
-> Application handler
-> Domain aggregate/entity
-> Domain repository contract
<- Infrastructure EF repository
-> SQLite (default) or PostgreSQL
```

The Application layer owns use-case orchestration. Infrastructure implements contracts; it does not define business policy.

4. Generated tree examples

Example A: Simple fullstack with WPF-UI and seed data

Command:

```
aspgen new Library --app fullstack --simple --seed:dummy --theme:wpfui --output ./Library
aspgen add entity Book title:string author:string pages:int published:date available:bool
--project ./Library
```

Tree:

```
Library/
  Library.sln
  .aspgen/manifest.json
  src/
    WebApi/
      WebApi.csproj
      Program.cs
      Data/
        AppDbContext.cs
        DatabaseSeeder.cs
        Models/Book.cs
        Features/Book/BookEndpoints.cs
    Desktop/
      Library.Desktop.csproj
      Modules/Book/
        BookModule.cs
        Models/BookRow.cs
        Services/BookStore.cs
        ViewModels/BookViewModel.cs
        Views/BookView.xaml
```

Best when: the product is mostly forms and CRUD, the team wants a fast vertical slice, and domain complexity is still low.

Representative API shape:

```
group.MapGet("/", async (AppDbContext db, CancellationToken ct) =>
Results.Ok(await db.Books.AsNoTracking().ToListAsync(ct)));
```

Example B: DDD fullstack

Command:

```
aspgen new Commerce --app fullstack --backend:ddd --seed dummy --output ./Commerce
aspgen add entity Order customerName:string total:decimal submitted:datetime paid:bool --project
./Commerce
```

Tree:

```
Commerce/
  Commerce.sln
  .aspgen/manifest.json
  src/
    Domain/Entities/Order.cs
    Application/Features/Order/
      CreateOrderCommand.cs
      CreateOrderHandler.cs
      UpdateOrderCommand.cs
      GetOrdersQuery.cs
      DeleteOrderCommand.cs
      OrderValidator.cs
    Infrastructure/
      Persistence/AppDbContext.cs
      Persistence/OrderRepository.cs
    WebApi/
      Features/Order/OrderEndpoints.cs
      Seeding/DatabaseSeeder.cs
    Desktop/Modules/Order/
      OrderModule.cs
      Services/OrderStore.cs
      Views/OrderView.xaml
```

Best when: the domain has invariants, a meaningful language, lifecycle rules, or multiple workflows that should not be hidden behind generic CRUD.

Representative handler shape:

```
public sealed class CreateOrderHandler(IOrderRepository repository)
: IHandler<CreateOrderCommand, OrderResponse>
{
    public async Task<Result<OrderResponse>> HandleAsync(
        CreateOrderCommand request, CancellationToken ct)
    {
        var order = new Order(request.CustomerName, request.Total,
            request.Submitted, request.Paid);
        await repository.AddAsync(order, ct);
        await repository.SaveChangesAsync(ct);
        return Result<OrderResponse>.Success(OrderResponse.From(order));
    }
}
```

Example C: Add UI after the server exists

Commands:

```
aspngen new Operations --app webapi --backend ddd --output ./Operations
aspngen add entity Person name:string active:bool --project ./Operations
aspngen add ui --framework wpf --theme:wpfui --project ./Operations
aspngen add module Reports --project ./Operations
```

Resulting change:

```
Before: Operations.sln -> Domain, Application, Infrastructure, WebApi
After: Operations.sln -> Domain, Application, Infrastructure, WebApi, Desktop
■■■ Desktop/Modules/Reports
```

Best when: backend work is already underway, a desktop workflow is a later release, or different teams own server and desktop delivery.

Representative Prism registration:

```
public void RegisterTypes(IContainerRegistry registry)
{
    registry.RegisterSingleton<IReportsStore, ReportsStore>();
    registry.RegisterForNavigation<ReportsView>();
}
```

5. Choosing the backend profile

Choose --simple when

- CRUD is the dominant behavior.
- The model has few invariants.
- A small team values a compact solution.
- You want direct EF Core endpoints and minimal ceremony.

Choose --backend ddd when

- The domain language is a competitive or operational asset.
- Invariants must be enforced in code, not only at the API boundary.
- Multiple use cases act on the same aggregate.
- You need repository contracts and explicit CQRS handlers.

Choose no backend profile when

- You are creating a host or architecture baseline first.
- You want to add individual features before committing to entity CRUD.
- The project is being used as a platform shell rather than a data application.

6. What each generated layer owns

Layer	Owns	Must not own
Domain	Rules, entities, aggregate behavior, value objects	HTTP, EF Core, WPF, configuration
Application	Use cases, handlers, validation, DTO records	UI controls, database provider details
Infrastructure	EF Core, SQLite/PostgreSQL, repositories, interceptors	Business decisions and endpoint mapping
WebApi	HTTP mapping, status codes, composition, OpenAPI	Domain logic and persistence algorithms
Desktop	Views, ViewModels, modules, navigation, API stores	Server-side invariants and database access
Manifest	Generator state and selected capabilities	Runtime business state

7. Decision checklist

Before generating:

1. Is this primarily CRUD or domain behavior?
2. Does the project need Web API, WPF, Blazor, or more than one host?
3. Should development data be present at startup?
4. Is the WPF theme a design requirement or a later option?
5. Will the team add capabilities incrementally?

After generating:

1. Inspect .aspgen/manifest.json.
2. Confirm every generated file is below an owning project root.
3. Confirm the solution lists every project.
4. Run go test ./... and template validation for generator changes.
5. Build the generated Web API/Desktop projects before committing the scaffold.

8. Pattern theory in one page

```

Gateway choice
↓
Profile grammar
↓
Owned project graph
↓
Vertical slice or module
↓
Tested runtime behavior

```

The generator follows a progressive-complexity principle: begin with the smallest architecture that preserves the next decision. Clean Architecture protects dependency direction. DDD protects language and invariants. CQRS protects use-case clarity. Prism protects desktop composition. Active Record protects delivery speed when the domain does not yet justify more layers.

9. Final recommendation

Start with `fullstack --simple` for a CRUD-first product, `fullstack --backend ddd` for a domain-first product, and add WPF incrementally when desktop workflows become real. Treat the generated tree as a map of ownership: if a file has no clear project, layer, or runtime reason, do not generate it.